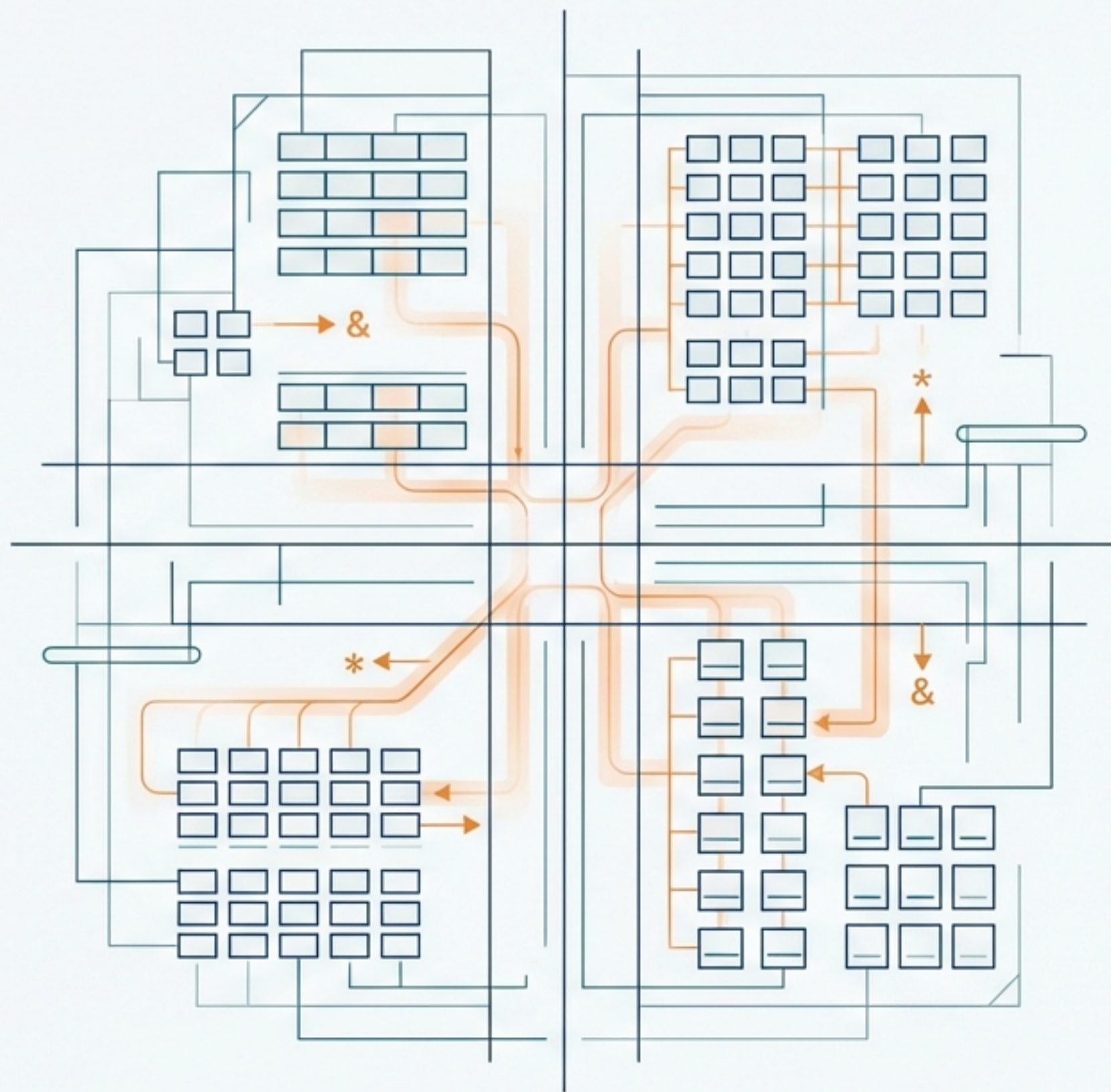


C语言核心：揭秘数组与指针的共生关系

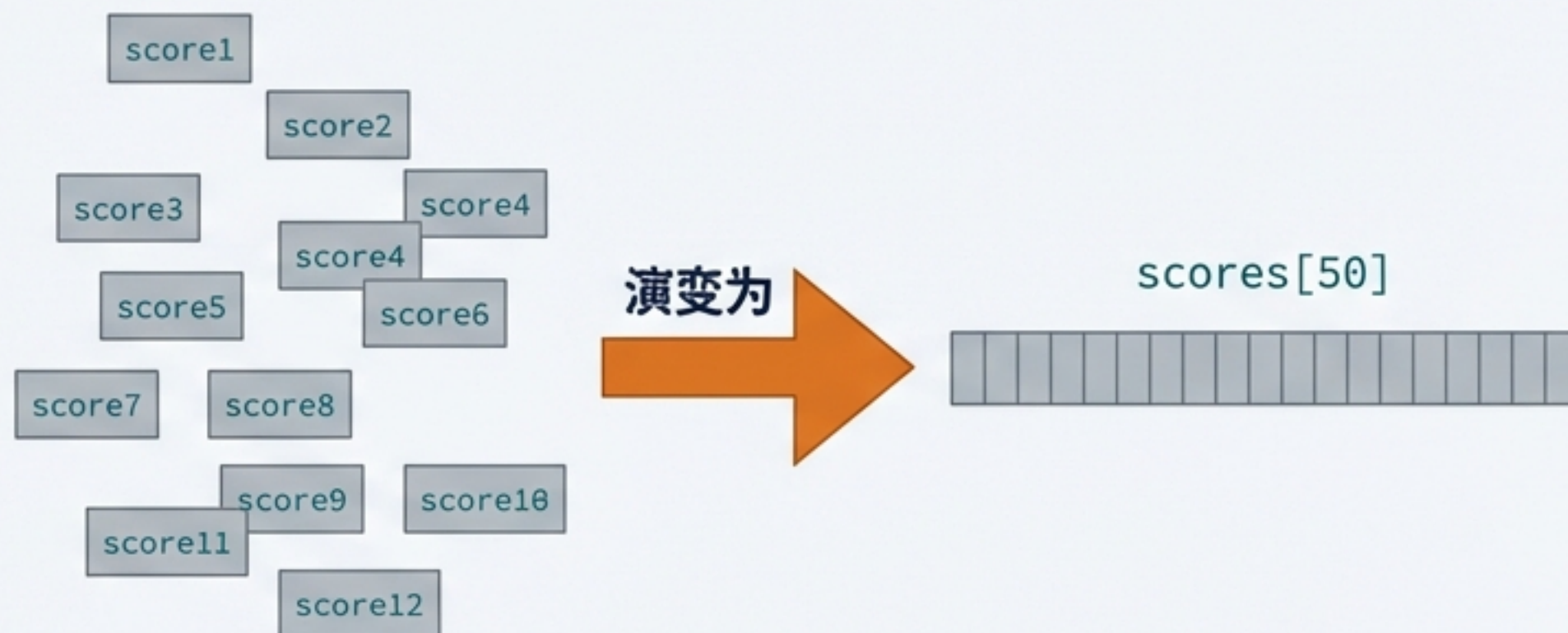
从语法表象到内存本质，彻底掌握数据操作的艺术



当单个变量力不从心

程序经常需要处理大量相关数据，例如月度降雨量或学生成绩。为每个数据点创建单独的变量（`score1`，`score2`，`score3...`）是低效且无法扩展的。

解决方案：数组（Arrays）——一种将相同类型的元素集合在一起的有序容器。



低效的方式

```
// 低效的方式  
int score1, score2, score3, ..., score50;
```

高效的数组方式

```
// 高效的数组方式  
int scores[50];
```


创建并赋予初始生命

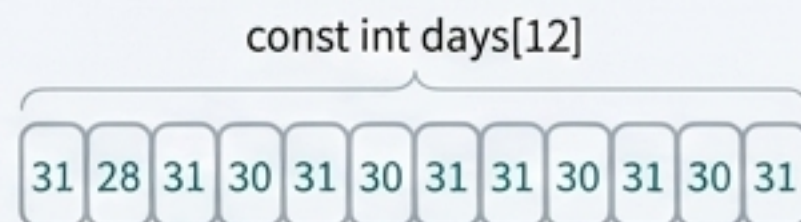
使用以逗号分隔的值列表（用花括号括起来）来初始化数组。

最佳实践：使用 `#define` 或 `const` 定义数组大小，便于维护。

关键规则：如果初始化列表中的值少于数组元素个数，编译器会将剩余的元素自动初始化为0。如果未初始化，数组元素将包含垃圾值。

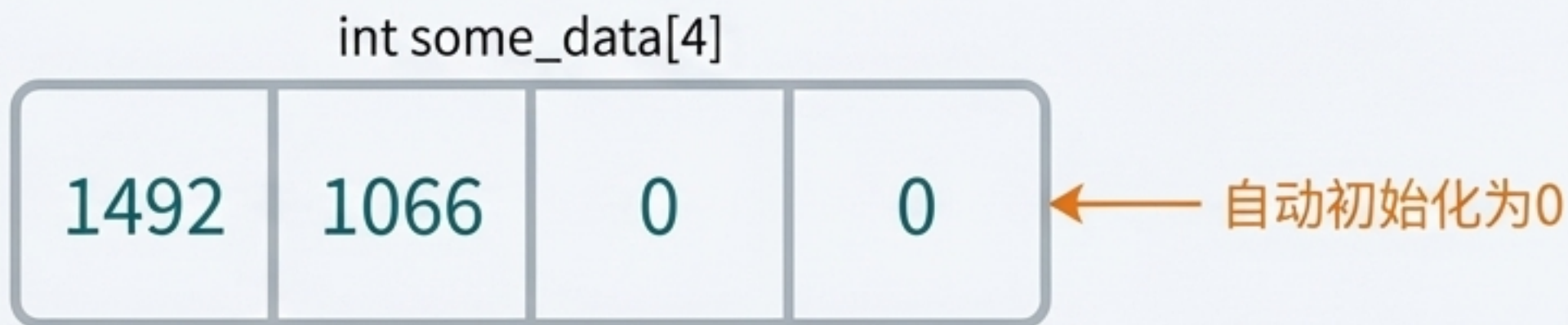
完整初始化

```
// 完整初始化
#define MONTHS 12
const int days[MONTHS] = {31, 28, 31, 30, ...};
```



部分初始化

```
// 部分初始化
#define SIZE 4
int some_data[SIZE] = { 1492, 1066 };
// some_data[2] 和 some_data[3] 将被初始化为 0。
```

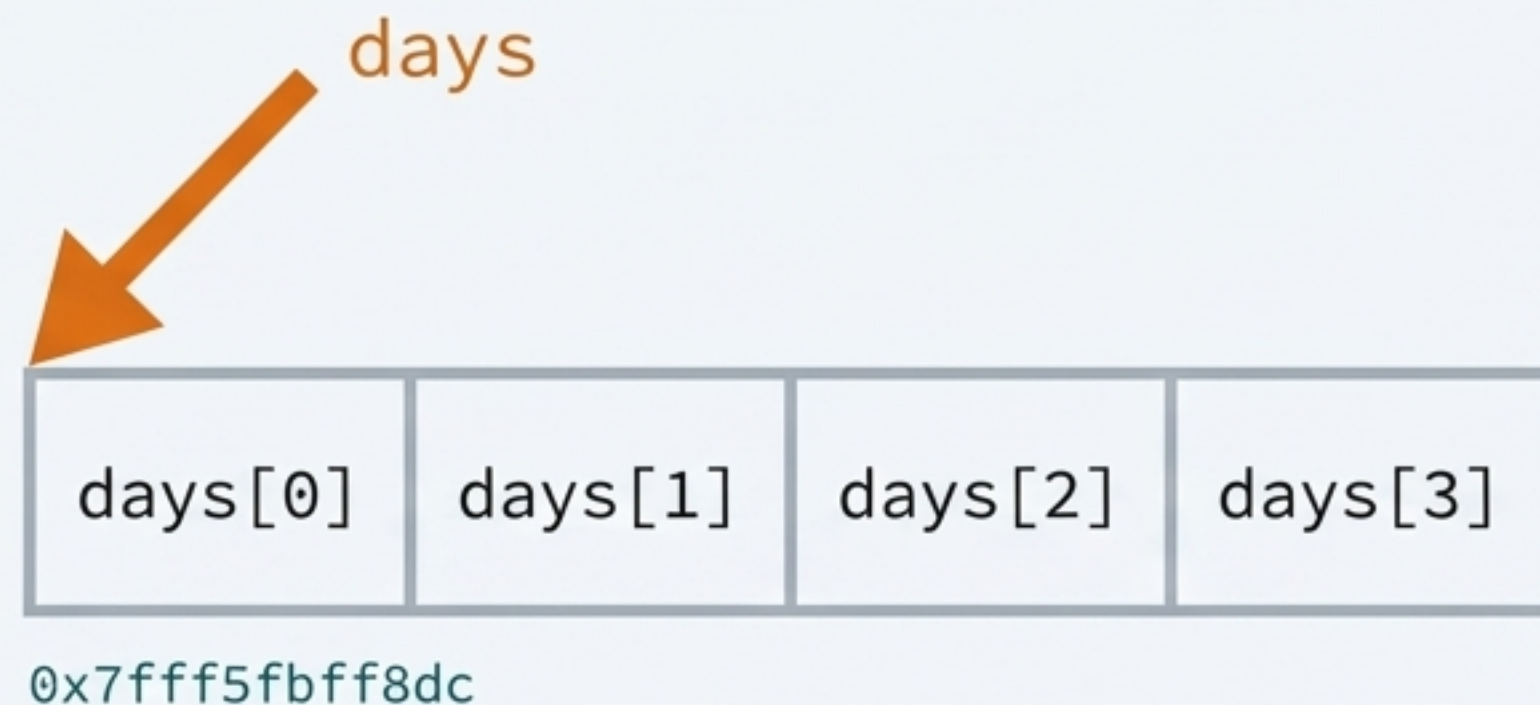


核心要点：初始化是数组的基石。部分初始化时，未指定元素默认为0。

数组名，不仅是一个名字

这是理解数组与指针关系的第一条线索：数组名本身并不是数组的容器，而是其**首元素的内存地址**。

这个地址是一个**常量**，在程序运行期间不可更改。你不能做 `days++` 这样的操作。



```
// 如果 flizny 是一个数组，那么下面的语句成立：  
flizny == &flizny[0];
```

核心要点：数组名是其首元素的地址。它是一个常量指针。

指针：赋予你直接操控地址的能力

指针是一种特殊的变量，它不存储数据值，而是存储另一个变量的**内存地址**。

1. 取址 (&)

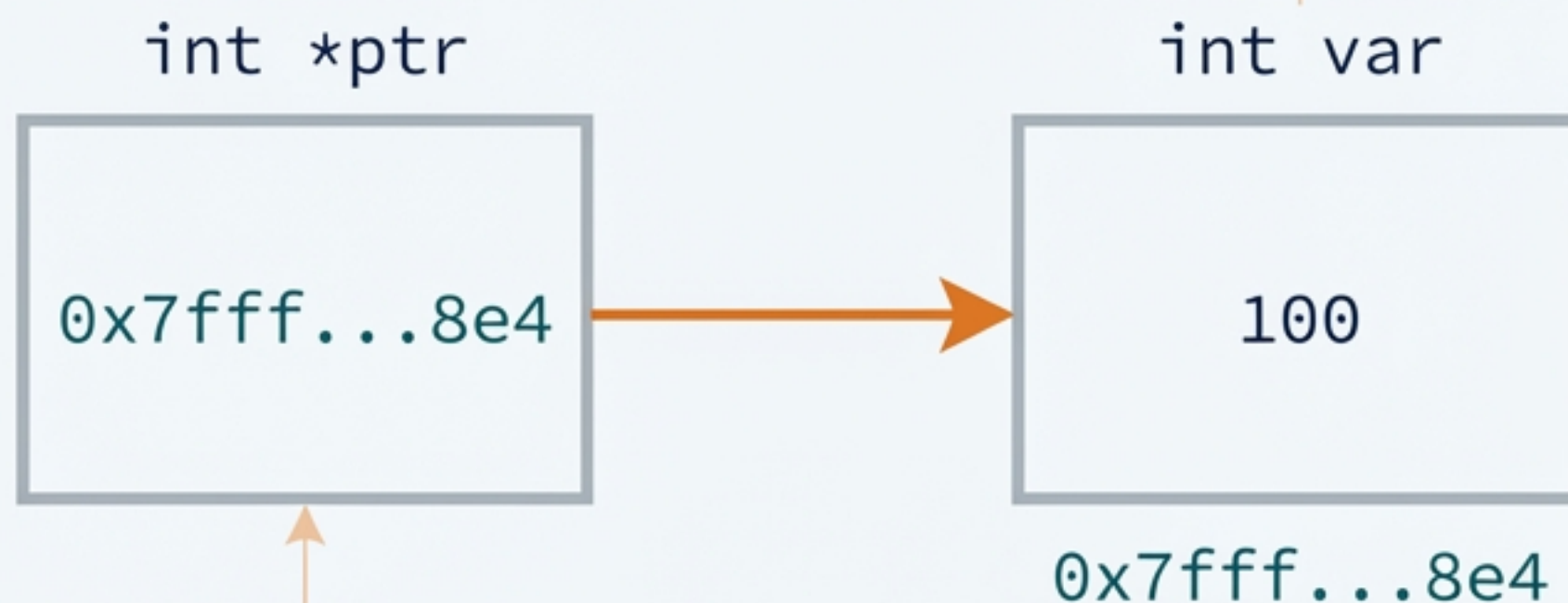
获取一个变量的地址并存入指针。

```
ptr = &var;
```

2. 解引用 (*)

访问指针所指向地址中存储的值。

```
value = *ptr;
```



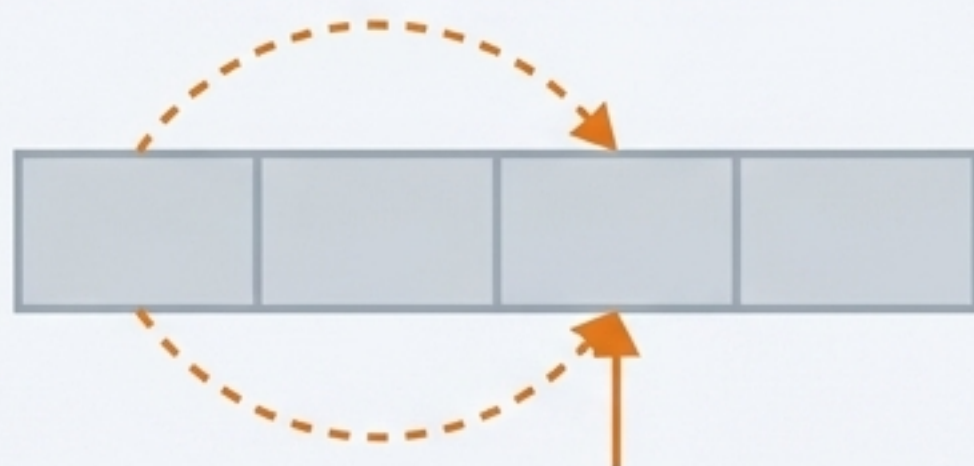
`days[i]` 只是 `\*(days + i)` 的“语法糖”

C语言标准在描述数组表示法时，直接使用了指针。ar[n] 的定义就是 $*(ar + n)$ 。这个表达式的含义是：“移动到内存地址 ar，从此地址前进 i 个元素单元，然后取出该位置的值。”

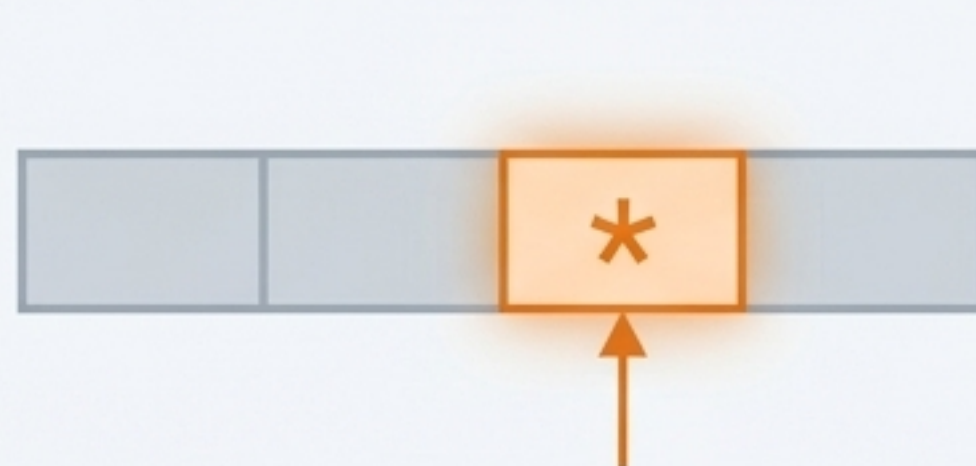
1. days: 数组首元素的地址



2. days + i: 地址移动 i 个元素单元



3. $*(days + i)$: 解引用，获取该地址的值



数组表示法 (Array Notation)

`days[i]`

优点：简洁，直观，易于阅读。

指针表示法 (Pointer Notation)

`*(days + i)`

优点：揭示了底层内存操作的本质。

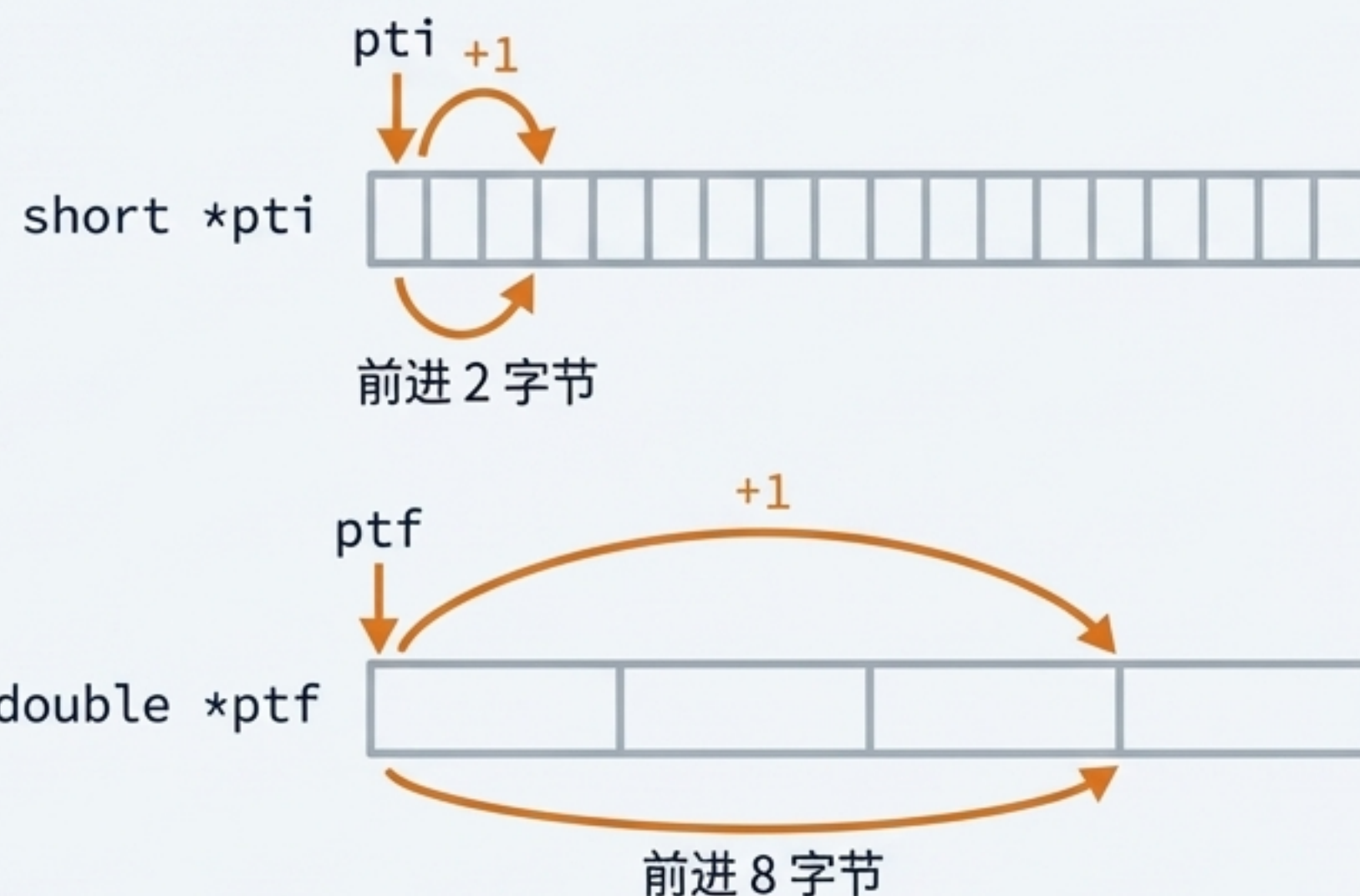
指针的“+1”为何与众不同？

- 在C中，给一个指针加1，实际增加的地址值是一个存储单元的大小。
- 编译器知道指针的类型。`+ 1`意味着移动到下一个元素的地址，而不是下一个字节的地址。
- 这就是为什么必须声明指针所指向的对象类型的原因。

```
short  dates[SIZE];  
short *pti = dates;
```

```
double bills[SIZE];  
double *ptf = bills;
```

```
// pti + 1 将地址增加 sizeof(short) 即 2 字节  
// ptf + 1 将地址增加 sizeof(double) 即 8 字节
```



你传递的不是数组，而是它的地址

- 当数组名作为函数参数时，传递的是一个指针——即数组首元素的地址。函数接收的是这个地址的一个副本。
- 这解释了为什么在函数内部对数组参数使用 **sizeof** 运算符，得到的是指针的大小，而不是整个数组的大小。

在 main() 中

```
int main(void) {  
    int marbles[10] = { ... };  
    // sizeof marbles 在这里是 10 * sizeof(int)  
    printf("The size of marbles is %zd bytes.\n",  
sizeof marbles);  
    sum(marbles, 10);  
}
```

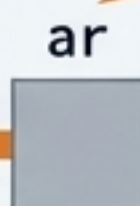
输出: The size of marbles is 40 bytes.



在 sum() 函数中

```
int sum(int ar[], int n) {  
    // ar 实际上是一个指针  
    // sizeof ar 在这里是指针的大小  
    printf("The size of ar is %zd bytes.\n",  
sizeof ar);  
}
```

输出: The size of ar is 8 bytes.



核心要点：向函数传递数组是高效的（避免了大数据拷贝），但也意味着函数可以直接修改原始数组。

const: 一个“君子协定”

- 当函数只需要读取数组数据而不应修改它时，应在函数原型和定义中使用关键字 `const` 来声明指针形参。
- `const` 告诉编译器，该函数应将数组视为只读。任何在函数内尝试修改数组的操作（如 `ar[i]++`）都会导致编译错误。
- 这并不是要求原始数组是常量，而是函数本身承诺不会修改它。

允许修改数据

```
void mult_array(double ar[], int n,  
double mult);
```

保护数据不被修改



```
void show_array(const double ar[],  
int n);
```

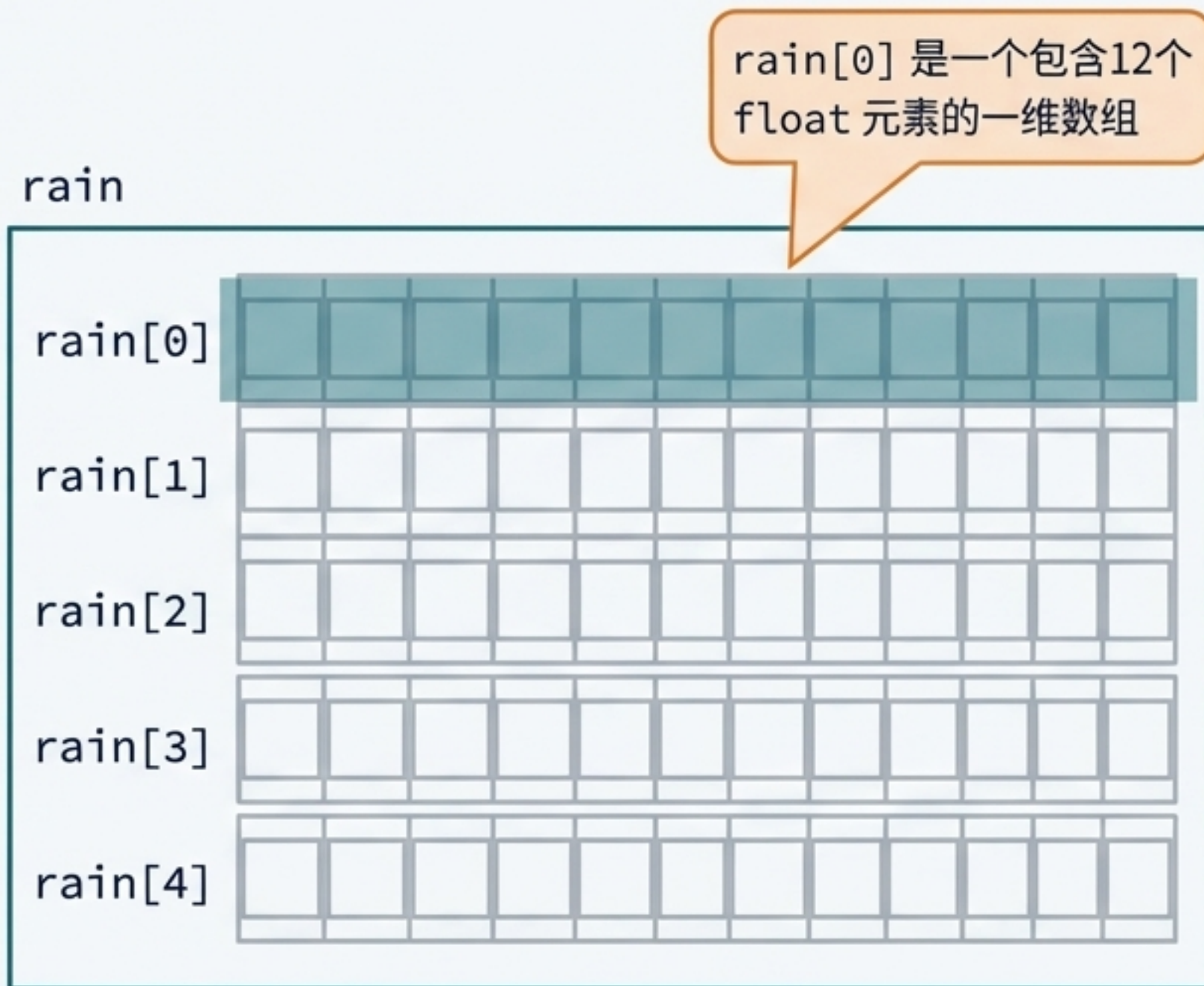
核心要点：用 `const` 保护你的数据，让代码意图更清晰、更安全。

数组的数组：数据从线到面

- 一个二维数组本质上是一个一维数组，其每个元素是另一个一维数组。
- `float rain[5][12]`；声明了一个包含5个元素的数组，其中每个元素本身是一个包含12个 `float` 值的数组。

指针视角

- `rain` 是其首元素 `rain[0]` 的地址。
- `rain[0]` 本身是一个包含12个 `float` 的数组。
- 因此，`rain` 是一个指向“包含12个float的数组”的指针。其类型为 `float (*)[12]`。



如何教会函数处理“表格”数据

将二维数组传递给函数时，函数需要知道如何计算行与行之间的地址偏移。

传统C方法（Pre-C99）

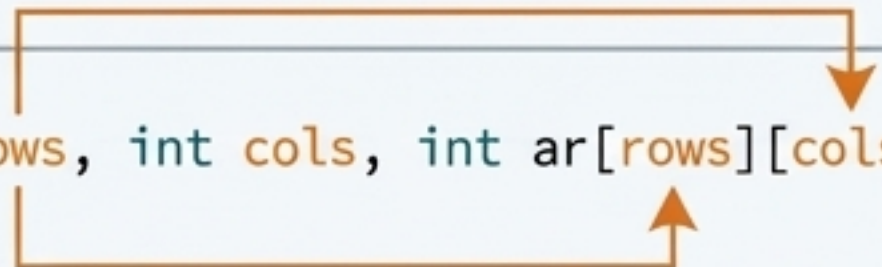
函数声明必须指定除第一维之外的所有维度的大小。编译器需要这个信息来进行指针运算（`ar + 1` 需要知道一行有多少字节）。

```
#define COLS 4  
void sum_rows(int ar[][COLS], int rows);
```

现代C方法（C99/C11 VLA）

可以将维度作为变量传递，形参的顺序很重要。

```
int sum2d(int rows, int cols, int ar[rows][cols]);
```



核心要点：

核心要点：传统方法中，除第一维外，其他维度必须在函数签名中指定，因为编译器需要它来进行指针运算。

C99/C11带来的灵活性与便利

变长数组 (Variable-Length Arrays - VLA)

- 允许使用变量来定义数组的维度。数组的大小在运行时确定，而不是编译时。
- **注意：**VLA一旦创建，其大小就固定了。“变”指的是创建时可以使用变量。

```
// 变长数组 (VLA)
int rs = 3;
int cs = 10;
int cs = 10;
int varr[rs][cs]; // 在运行时创建了一个 3x10 的数组
```

复合字面量 (Compound Literals)

- 允许创建匿名的、临时的数组，非常适合作为函数参数，无需预先声明一个命名数组。

```
int total;
total = sum((int []){4, 4, 4, 5, 5, 5}, 6);

// 复合字面量 (Compound Literal)
int total;
total = sum((int []){4, 4, 4, 5, 5, 5}, 6);
```


指针操作速查表：规则与禁区

允许的操作 ✓

- 赋值: `ptr1 = urn;` 或 `ptr2 = &urn[2];`
- 解引用: `*ptr1`
- 加/减整数: `ptr1 + 4`, `ptr3 - 2`
- 递增/递减: `ptr1++`, `--ptr2`
- 指针求差: `ptr2 - ptr1` (得到元素间距离)
- 比较: `if (ptr1 < ptr2)`

禁止或危险的操作 ✗

- 指针相加: `ptr1 + ptr2` (无意义)
- 指针乘除
- 解引用未初始化的指针 (致命错误!)

⚠ 代码警示

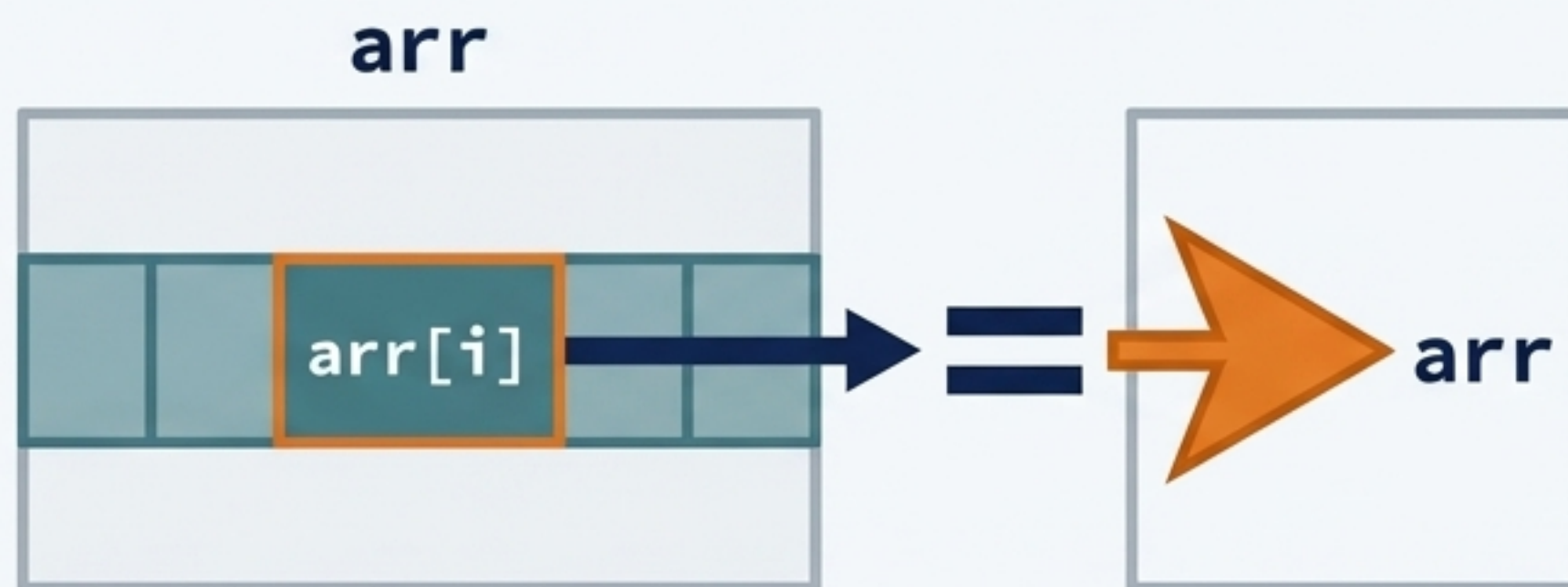
```
int * pt;    // pt 包含一个随机的垃圾地址
*pt = 5;     // 灾难! 试图在未知内存位置写入5。
             // 可能导致程序崩溃或数据损坏。
```

核心原理：创建指针时，系统只为指针本身分配了内存，并未为它将要指向的数据分配内存。

核心洞察：数组与指针的一体两面

1. 数组名是其首元素的常量地址。

- days 等价于 &days[0]，但 days 不可被修改。



2. arr[i] 是 *(arr + i) 的一种更易读的写法。

数组索引从根本上说是一种指针运算。

$$\text{arr}[i] == *(\text{arr} + i)$$

3. 向函数传递数组，本质是传递一个指针。

- 这解释了效率和 sizeof 的行为。

4. 指针的算术运算是基于其所指类型的大小。

- ptr + 1 会跳过整个 sizeof(*ptr) 字节。



指针不是魔法，而是C语言与硬件沟通的桥梁。

理解了指针，你便理解了C如何将高级逻辑映射到机器的物理内存之上。你掌握的不仅是语法，更是对计算本质的洞察。